

CS 440: Introduction to Artificial Intelligence

Assignment 1: Fast Trajectory Planning

Mary Grlic
Soraya Mosavi

Due: February 22, 2026

Part 0

We implemented grid generation using a randomized DFS-style traversal. Each cell is assigned as blocked with probability 0.3 and unblocked otherwise. We generated 50 distinct 101×101 grid maps and saved them to files for consistent experimentation. The maps can be loaded and visualized for testing the search algorithms.

Part 1

Initially, the agent does not know which cells are blocked so it assumes all cells are free/unblocked. Since the agent is assumed to use the Manhattan distance, it computes the shortest path to another cell given there are no obstacles. In figure 8, moving east and north both have equal f -values, and A* must break the tie between the two nodes. To do so, the algorithm would favor the cell with a larger g -value, as stated in the detailed project description.

An agent in a finite gridworld will either reach its target or discover that this is impossible in finite time. Given a grid is finite, this means that there is a finite number of cells. In the search algorithm, the agent reaches each cell and updates its knowledge on each blocked cell. A cell is thus only reached and discovered once, so if the number of cells is finite, the number of times each cell is reached must be finite. Each A* search expands into finite states, so the agent will either reach the target or discover that this is impossible in a finite time.

Part 2

We compared repeated forward A* using two tie-breaking rules when f -values are equal: favoring smaller g vs favoring larger g . On our 50 generated 101×101 maps, both approaches solved 26 maps. However, for the 26 maps solved by both, the large- g tie breaker expanded fewer nodes on every single map (26 wins, 0 ties). On average, small- g expanded 267,980.5 nodes while large- g expanded 9,261.08 nodes. This suggests that when f ties occur, preferring larger g tends to reduce search effort in this repeated replanning setting, likely because it commits more strongly to deeper partial paths instead of exploring many shallow alternatives.

Part 3

For Part 3, we compared Repeated Forward A* and Repeated Backward A* on the same 50 generated 101×101 maps, using the large- g tie-breaking rule (since it performed better in Part 2). Both algorithms were able to solve 26 out of the 50 maps, and they solved the exact same 26 maps. However, there was a big difference in how many nodes they expanded. On average, Forward A* expanded about 9,261 nodes, while Backward A* expanded around 129,930 nodes. Forward A* expanded fewer nodes on all 26 maps that were solved by both algorithms, and there were no ties. This shows that, in our experiments, planning from the agent's current position toward the goal

(Forward A*) was much more efficient than planning backward from the goal to the agent. Even though both methods eventually found a path on the same maps, Forward A* required significantly less search effort.

Part 4

Manhattan distances are consistent in gridworlds in which the agent can move only in the four main compass directions. The Manhattan distance is defined as the distance between two points on a grid and it is assumed that an agent can only move up, down, left, or right. Manhattan distance is the fastest way for an agent to move since the agent cannot move diagonally. The shortest distance will always be found with the Manhattan distance. Thus, the Manhattan distance is consistent in a gridworld.

A heuristic is consistent if and only if, for every node n and every successor n' of n , the estimated cost from a node to its goal is always less than or equal to the cost of moving to a neighboring node plus the estimated cost from the neighbor to the goal. Mathematically, this can be represented by the following equation:

$$h(n) \leq c(n, a, n') + h(n')$$

such that $h(n) = 0$ and $n.\text{State} = \text{goal}$.

In adaptive A* search, heuristic values are updated as follows:

$$h_{\text{new}}(n) = g^*(\text{goal}) - g(n)$$

Under the assumption of shortest-path distances, i.e., the algorithm prioritizes the shortest path, the optimal cost from n to the goal is always less than or equal to the cost of moving from n to n' plus the cost from n' to the goal. This implies that

$$h_{\text{new}}(n) \leq c(n, n') + h_{\text{new}}(n')$$

Even if the action cost increases, the values of $h_{\text{new}}(n)$ will always be less than or equal to the right side of the equation. Thus the adaptive A* method preserves consistency.

Part 5

For Part 5, we ran Adaptive A* and Repeated Forward A* (using the large- g tie breaker) on the same 50 generated 101×101 maps. Both algorithms solved 26 maps, and they solved the same exact 26 maps. When we compared the number of expanded nodes, Adaptive A* usually did a little better. On average, Forward A* expanded 9261.08 nodes while Adaptive A* expanded 9155.62 nodes. Adaptive A* expanded fewer nodes on 25 of the 26 maps that both algorithms solved

(Forward only won once, with no ties). This makes sense because Adaptive A* reuses information from previous searches by updating its heuristic values, so later replans tend to expand fewer states.

Part 6

For Part 6, we can test whether the performance difference between Repeated Forward A* and Adaptive A* is statistically significant. The null hypothesis (H_0) would be that there is no real difference in the number of expanded nodes between the two algorithms, and that any observed difference is due to random variation in the selected 50 maps. The alternative hypothesis (H_1) would be that Adaptive A* expands significantly fewer nodes than Repeated Forward A*.

Since both algorithms are tested on the same set of 50 maps, we can treat the data as paired samples and use a paired t -test. For each map, we compute the difference in the number of expanded nodes (Forward - Adaptive). Then we calculate the mean and standard deviation of these differences. The paired t -test would determine whether the average difference is significantly different from zero.

If the resulting p -value is smaller than a chosen significance level (for example, $\alpha = 0.05$), we would reject the null hypothesis and conclude that the performance difference is statistically significant. Otherwise, we would conclude that the observed difference could be due to sampling noise.